

Divide and Conquer

Divide and Conquer

Divide and Conquer is a strategy that solves a problem by:

- 1) Breaking the problem into subproblems that are themselves smaller instances of the same type of problem.
- 2) Recursively solving these subproblems.
- 3) Appropriately combining their answers.

Searching

Search

Input: A list of numbers, sorted in increasing order, $A = [a_0, a_1, a_2, a_3, \dots, a_{n-1}]$ and a number x .

Goal: Return a position of x in A if $x \in A$ or return a statement that x is not in the list.

- ☐ How would a four-year-old do this?
- ☐ How would a 2nd grader do this?
- ☐ How would you do this?

BinarySearch

BinarySearch(*A*, *x*, *min*, *max*)

Input: An array of n sorted numbers A , number x , integer indices lo , and hi . (Initially $lo = 1$ and $hi = n$)

Output: An index of x in A if $x \in A$, or -1 if $x \notin A$.

```
1: if  $hi < lo$  then  
    return  $-1$   
2: else  
3:    $mid \leftarrow \lfloor (hi+lo)/2 \rfloor$   
4:   if  $x < A[mid]$  then  
       return BinarySearch(A, x, lo,  $mid - 1$ )  
5:   else if  $x > A[mid]$  then  
       return BinarySearch(A, x,  $mid + 1$ , hi)  
6:   else  
       return mid
```

BinarySearch

We must ask ourselves two questions about this algorithm:

- ❑ Is it correct?
- ❑ What is its running time?

At each stage we divide the problem in half and it takes constant time to “combine” solutions:

$$T(n) = T(n/2) + O(1)$$

Later, we will learn how to “solve” this recurrence relationship into something like $O(\log n)$

BinarySearch - Correctness

We must ask ourselves two questions about this algorithm:

- ❑ Is it correct?
- ❑ What does "correct" mean?
 - ❑ Look at the problem description
- ❑ The program must:
 - ❑ Always halt (rather than looping forever)
 - ❑ Return -1 if x does not exist in the search range
 - ❑ Return an index of x if it does exist in the search range

BinarySearch - Correctness

While we could prove all requirements for correctness in one go, let's break it down

Lemma 1

BinarySearch always halts

Sketch of proof

1. On every iteration, either lo increases by at least 1, or hi decreases by at least 1
2. Therefore, the quantity $n = hi - lo + 1$ is strictly decreasing
3. Therefore, at some point we must reach the base case $hi < lo$

Theorem 1

If x is not an element of the list, BinarySearch will return -1 .

Proof (by contradiction)

1. Lemma 1 tells us that *BinarySearch* always halts, so it must return some value r
2. Suppose $x \notin A$, that is $\forall i, A[i] \neq x$ and yet $r \neq -1$
3. The only step where the algorithm terminates on a value other than -1 is step 6
4. But this only happens if it is neither the case that $x < A[mid]$ nor $x > A[mid]$
5. Therefore, $x = A[mid]$. Contradiction with line 2 of the proof

BinarySearch - Correctness

While we could prove all requirements for correctness in one go, let's break it down

Theorem 2

If $x \in A$, Binary Search returns an index of x in A

Proof by induction on $n = hi - lo + 1$, shown in class